

Classification - Partie 1

K-Nearest Neighbors et Arbres de Décision

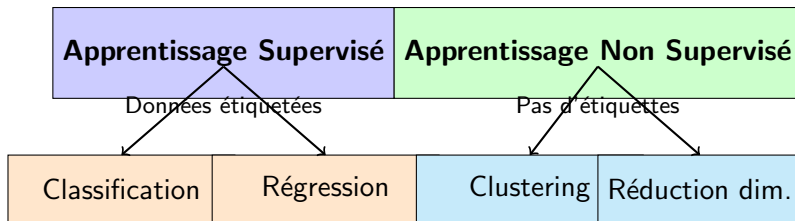
Meyssem Bouzaïen

Année universitaire: 2025 - 2026

Plan du Cours

- 1 Qu'est-ce que la Classification ?
- 2 K-Nearest Neighbors (KNN)
- 3 Arbres de Décision
- 4 Métriques d'Évaluation
- 5 Exemple Pratique Complet
- 6 Conseils Pratiques

Rappel : Types d'Apprentissage



Aujourd'hui

On se concentre sur la **Classification**

Qu'est-ce que la classification ?

La classification consiste à **prédire une catégorie** (classe) pour de nouvelles observations, à partir d'exemples étiquetés.

Input (Features) :

- Caractéristiques mesurables
- Variables descriptives
- Exemple : taille, poids, couleur...

Output (Classe) :

- Catégorie prédite
- Variable discrète
- Exemple : chat, chien, oiseau

Exemples de Classification

Application	Features (X)	Classes (y)
Email Spam	Mots, expéditeur, liens	Spam / Non-spam
Diagnostic médical	Symptômes, analyses	Malade / Sain
Reconnaissance d'images	Pixels de l'image	Chat / Chien / Oiseau
Crédit bancaire	Revenu, âge, historique	Accordé / Refusé
Sentiment analysis	Texte d'un avis	Positif / Négatif / Neutre
Reconnaissance de fleurs	Taille sépales/pétales	Setosa / Versicolor / Virginica

La classification est partout !

Classification Binaire vs Multi-classes

Classification Binaire

2 classes seulement

- Oui / Non
- Spam / Ham
- Malade / Sain
- Positif / Négatif

Classification Multi-classes

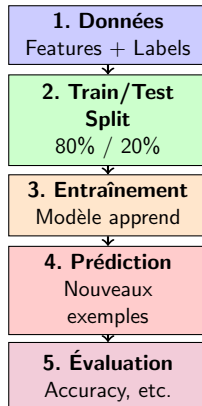
Plus de 2 classes

- 3 types de fleurs Iris
- 10 chiffres (0-9)
- 26 lettres (A-Z)
- 1000 catégories d'objets

Important

Aujourd'hui, on verra des algorithmes qui fonctionnent pour les deux !

Le Processus de Classification



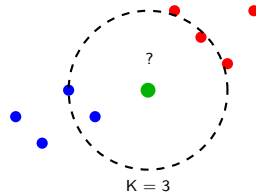
KNN : L'Algorithme le Plus Simple

Idée de base

“Dis-moi qui sont tes voisins, je te dirai qui tu es”

Principe :

- 1 Trouver les K plus proches voisins
- 2 Regarder leur classe
- 3 Vote majoritaire
- 4 Assigner la classe gagnante



KNN : Comment ça marche ? (1/2)

Étape 1 : Calculer la distance

Pour chaque point dans le dataset, calculer la distance avec le nouveau point.

Distance euclidienne (la plus courante) :

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Exemple

Point A = (1, 2) et Point B = (4, 6)

$$d = \sqrt{(4 - 1)^2 + (6 - 2)^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

KNN : Comment ça marche ? (2/2)

Étape 2 : Trier par distance

Garder les K points les plus proches

Étape 3 : Vote majoritaire

Compter les classes parmi les K voisins

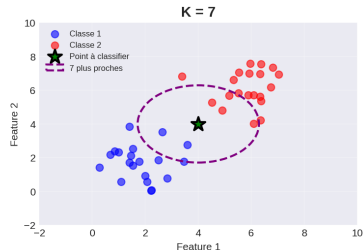
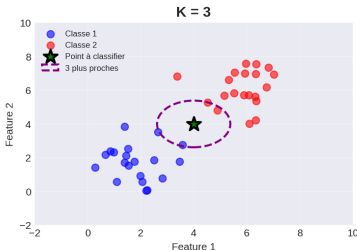
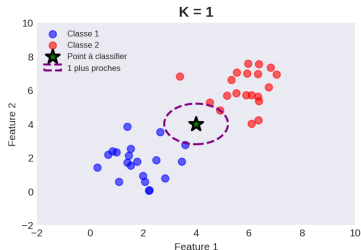
Exemple avec $K=5$

- Voisin 1 : Classe A (distance : 1.2)
- Voisin 2 : Classe A (distance : 1.5)
- Voisin 3 : Classe B (distance : 1.8)
- Voisin 4 : Classe A (distance : 2.1)
- Voisin 5 : Classe B (distance : 2.3)

Vote : A=3, B=2 → **Classe prédite = A**

Le Paramètre K : Impact

K-Nearest Neighbors: Impact du paramètre K



Conseil

- K trop petit → overfitting (bruit)
- K trop grand → underfitting (perd les détails)
- Valeurs courantes : $K = 3, 5, 7$ (nombres impairs pour éviter les égalités)

KNN avec Scikit-learn

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 1. Charger les donnees
iris = load_iris()
X = iris.data
y = iris.target

# 2. Train/Test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# 3. Creer le modele KNN
knn = KNeighborsClassifier(n_neighbors=5)

# 4. Entraîner
knn.fit(X_train, y_train)

# 5. Predire
y_pred = knn.predict(X_test)

# 6. Evaluer
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy*100:.2f}%")
```

KNN : Avantages et Inconvénients

Avantages

- Simple à comprendre
- Pas d'entraînement
- Fonctionne bien pour petits datasets
- Pas d'hypothèses sur les données
- Multi-classes naturellement

Inconvénients

- Lent pour grandes données
- Sensible aux features non normalisées
- Sensible au bruit
- Besoin de choisir K
- Coûteux en mémoire

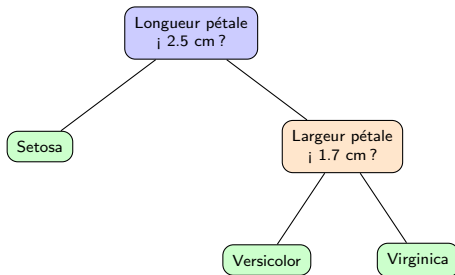
Quand utiliser KNN ?

Petits datasets, peu de features, besoin de simplicité

Arbres de Décision : Intuition

L'idée

Un arbre de décision pose une série de **questions** sur les features pour arriver à une décision.



Comme un organigramme de décisions !

Comment Construire un Arbre ?

Algorithme (simplifié)

- ① Choisir la meilleure question à poser
- ② Diviser les données selon cette question
- ③ Répéter pour chaque branche
- ④ Arrêter quand :
 - Tous les exemples ont la même classe
 - Profondeur maximale atteinte
 - Nombre minimum d'exemples atteint

Question clé

Comment choisir la **meilleure** question ?

→ Celle qui sépare le mieux les classes

Critère de Division : Gini et Entropie

Mesures d'impureté

On veut des groupes **purs** (une seule classe par groupe)

Indice de Gini :

$$Gini = 1 - \sum_{i=1}^n p_i^2$$

- Gini = 0 → parfaitement pur
- Gini = 0.5 → mélange 50/50

Entropie :

$$H = - \sum_{i=1}^n p_i \log_2(p_i)$$

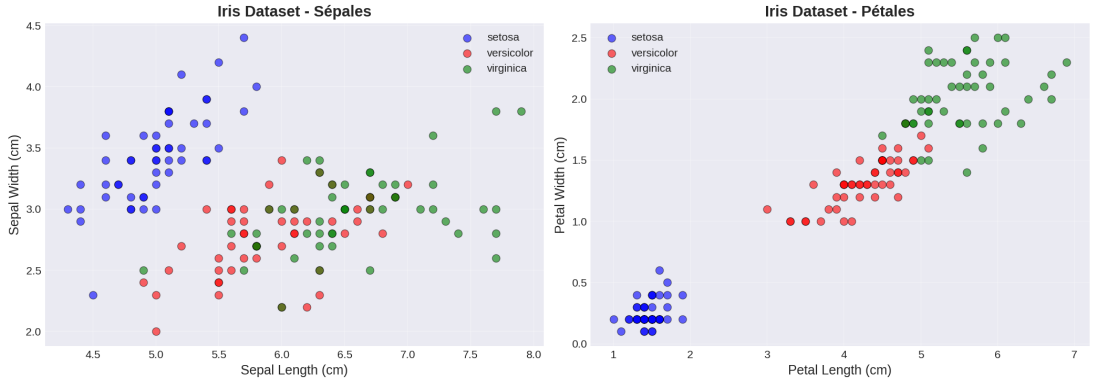
- H = 0 → parfaitement pur
- H élevé → désordre

Exemple

Groupe avec 10 bleus et 0 rouge → Gini = 0 (pur !)

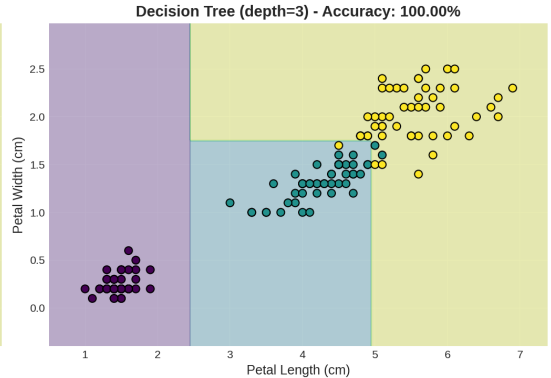
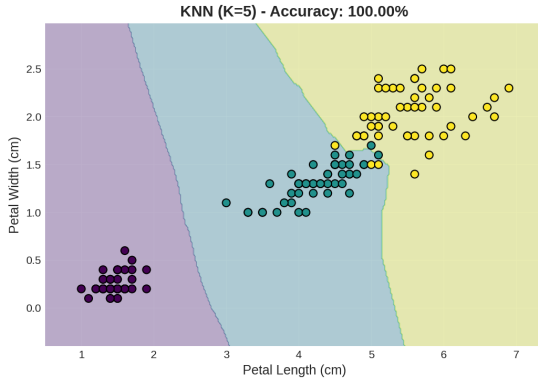
Groupe avec 5 bleus et 5 rouges → Gini = 0.5 (impur)

Exemple Visuel : Dataset Iris



Les pétales séparent mieux les 3 espèces que les sépales !

Frontières de Décision : KNN vs Arbre



KNN : Frontières lisses — **Arbre** : Frontières rectangulaires

Arbres de Décision avec Scikit-learn

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 1. Charger les donnees
iris = load_iris()
X = iris.data
y = iris.target

# 2. Train/Test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# 3. Creer l'arbre de decision
tree = DecisionTreeClassifier(
    max_depth=3,          # Profondeur maximale
    min_samples_split=2   # Min exemples pour diviser
)

# 4. Entraîner
tree.fit(X_train, y_train)

# 5. Predire
y_pred = tree.predict(X_test)

# 6. Evaluer
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy*100:.2f}%")
```

Contrôler la complexité de l'arbre

- **max_depth** : Profondeur maximale de l'arbre
 - Trop grand → overfitting
 - Trop petit → underfitting
- **min_samples_split** : Nombre minimum d'exemples pour diviser un nœud
 - Plus grand → arbre plus simple
- **min_samples_leaf** : Nombre minimum d'exemples dans une feuille
 - Évite les feuilles avec trop peu de données
- **criterion** : 'gini' ou 'entropy'
 - En pratique, peu de différence

Arbres : Avantages et Inconvénients

Avantages

- Facile à comprendre et visualiser
- Pas besoin de normalisation
- Gère features numériques et catégorielles
- Gère valeurs manquantes
- Rapide à entraîner et prédire
- Indique features importantes

Inconvénients

- Tendance à l'overfitting
- Instable (petits changements → arbre différent)
- Biais vers features dominantes
- Moins précis que d'autres méthodes

Solution

Les **Forêts Aléatoires** (cours suivant) combinent plusieurs arbres pour être plus robustes !

Pourquoi Évaluer ?

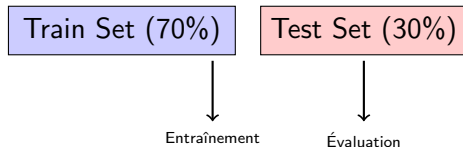
Question fondamentale

Comment savoir si notre modèle est **bon** ?

Règle d'or

JAMAIS évaluer sur les données d'entraînement !

→ Risque de surestimer les performances



Accuracy (Précision)

Définition

Pourcentage de prédictions correctes

$$Accuracy = \frac{\text{Nombre de prédictions correctes}}{\text{Nombre total de prédictions}}$$

Exemple

Sur 100 fleurs :

- 95 bien classées
- 5 mal classées

$$Accuracy = \frac{95}{100} = 0.95 = 95\%$$

Attention !

L'accuracy peut être trompeuse si les classes sont déséquilibrées !

Problème des Classes Déséquilibrées

Exemple : Détection de Fraude

Dataset : 1000 transactions

- 990 normales
- 10 frauduleuses

Modèle naïf : Prédire "Normal" pour tout

→ Accuracy = 99% !

Mais : 0% de fraudes détectées !

Solution

Utiliser d'autres métriques : Précision, Rappel, F1-Score

Matrice de Confusion

Définition

Tableau qui montre les prédictions vs la réalité

		Prédiction	
		Positif	Négatif
Réalité	Positif	VP	FN
	Négatif	FP	VN

- **VP** (Vrai Positif) : Prédit positif, c'est vrai
- **VN** (Vrai Négatif) : Prédit négatif, c'est vrai
- **FP** (Faux Positif) : Prédit positif, c'est faux (Erreur Type I)
- **FN** (Faux Négatif) : Prédit négatif, c'est faux (Erreur Type II)

Exemple de Matrice de Confusion

Classification de 100 emails

		Prédiction	
		Spam	Ham
Réalité	Spam	35	5
	Ham	3	57

- VP = 35 (spams bien détectés)
- VN = 57 (hams bien classés)
- FP = 3 (hams classés comme spam)
- FN = 5 (spams non détectés)

$$Accuracy = \frac{35+57}{100} = 92\%$$

Précision et Rappel

Précision (Precision)

Parmi les prédictions positives, combien sont vraies ?

$$Precision = \frac{VP}{VP+FP}$$

“Quand je dis spam, c'est vraiment du spam ?”

Rappel (Recall)

Parmi les vrais positifs, combien sont détectés ?

$$Recall = \frac{VP}{VP+FN}$$

“Est-ce que je détecte tous les spams ?”

Exemple précédent

$$Precision = \frac{35}{35+3} = 0.92 = 92\%$$

$$Recall = \frac{35}{35+5} = 0.875 = 87.5\%$$

F1-Score

Problème

Précision et Rappel peuvent être en conflit !

Trade-off

- Augmenter Précision → diminuer Rappel
- Augmenter Rappel → diminuer Précision

F1-Score : Moyenne harmonique

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

- F1-Score équilibre Précision et Rappel
- Utile quand les classes sont déséquilibrées

Calculer les Métriques avec Scikit-learn

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, classification_report

# Predictions
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

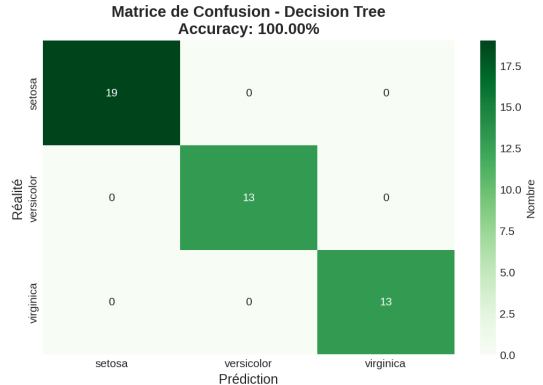
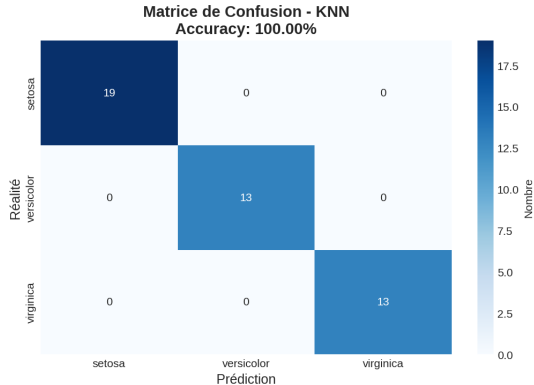
# Precision, Recall, F1
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
print(f"F1-Score: {f1:.2f}")

# Matrice de confusion
cm = confusion_matrix(y_test, y_pred)
print(cm)

# Rapport complet
print(classification_report(y_test, y_pred))
```

Matrices de Confusion : KNN vs Arbre



Les deux modèles obtiennent 100% de précision sur le test Iris !

Dataset Iris : Rappel

```
from sklearn.datasets import load_iris
import pandas as pd

# Charger
iris = load_iris()

# Créer un DataFrame pour visualiser
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

print(df.head())
print("\nClasses:", iris.target_names)
print("Features:", iris.feature_names)
```

- 150 fleurs
- 4 features (longueur/largeur sépales et pétales)
- 3 classes (Setosa, Versicolor, Virginica)

Comparaison KNN vs Arbre (1/2)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# Charger et separer
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=42)

# KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)

# Arbre de decision
tree = DecisionTreeClassifier(max_depth=3, random_state=42)
tree.fit(X_train, y_train)
y_pred_tree = tree.predict(X_test)
```


Comparaison KNN vs Arbre (2/2)

```
# Evaluer KNN
print("=== KNN ===")
print(f"Accuracy: {accuracy_score(y_test, y_pred_knn):.2f}")
print(classification_report(y_test, y_pred_knn,
                             target_names=iris.target_names))

# Evaluer Arbre
print("\n=== Decision Tree ===")
print(f"Accuracy: {accuracy_score(y_test, y_pred_tree):.2f}")
print(classification_report(y_test, y_pred_tree,
                             target_names=iris.target_names))
```

Résultats typiques :

- KNN : 95-98% accuracy
- Arbre : 93-96% accuracy

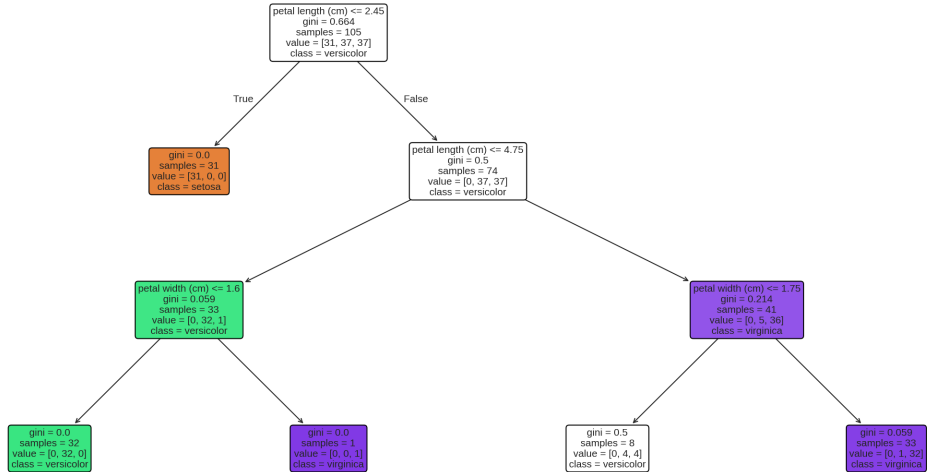
Visualiser l'Arbre de Décision

```
1 from sklearn.tree import plot_tree
2 import matplotlib.pyplot as plt
3
4 # Créer une grande figure
5 plt.figure(figsize=(20,10))
6
7 # Visualiser l'arbre
8 plot_tree(tree,
9           feature_names=iris.feature_names,
10          class_names=iris.target_names,
11          filled=True,
12          rounded=True,
13          fontsize=10)
14
15 plt.title("Arbre de Decision - Iris Dataset")
16 plt.savefig('decision_tree_iris.png', dpi=150, bbox_inches='tight')
17 plt.show()
```

Permet de comprendre comment l'arbre prend ses décisions !

L'Arbre de Décision Visualisé

Arbre de Décision - Dataset Iris (4 features)



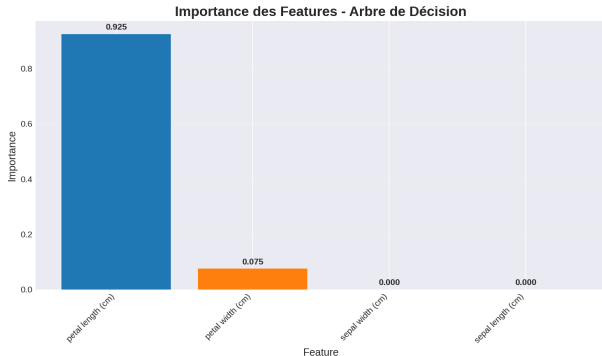
Importance des Features

```
import numpy as np
import matplotlib.pyplot as plt

# Obtenir importance
importances = tree.feature_importances_
features = iris.feature_names

# Trier
indices = np.argsort(
    importances)[::-1]

# Visualiser
plt.bar(range(len(importances)),
        importances[indices])
plt.xticks(range(len(importances)),
           [features[i] for i in indices])
plt.title("Importance Features")
plt.show()
```



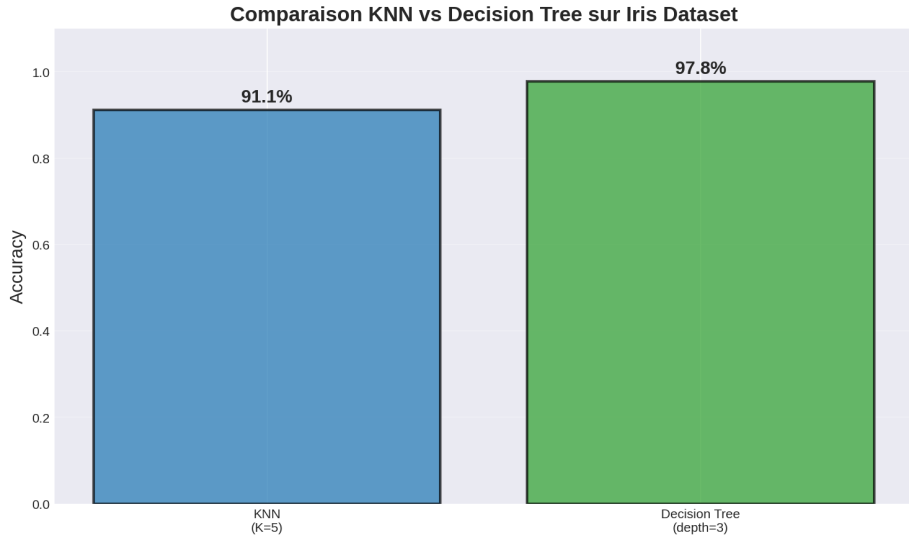
KNN vs Arbres : Quand Utiliser Quoi ?

Utilisez KNN si :	Utilisez Arbres si :
Dataset petit/moyen	Dataset moyen/grand
Peu de features (≤ 20)	Beaucoup de features
Données bien normalisées	Pas besoin de normaliser
Relations complexes non-linéaires	Besoin d'interprétabilité
Temps d'entraînement pas critique	Temps de prédiction important
Pas besoin d'explication	Besoin de comprendre les règles

En pratique

Essayez les deux et comparez !

Comparaison Finale : Performance



- ① **Toujours** diviser train/test
- ② **Normaliser** les données pour KNN
- ③ **Tester plusieurs valeurs** de K (KNN) ou max_depth (Arbres)
- ④ **Visualiser** les résultats (matrice de confusion)
- ⑤ **Ne pas** se fier uniquement à l'accuracy
- ⑥ **Vérifier** l'overfitting (accuracy train vs test)
- ⑦ **Comprendre** les erreurs du modèle

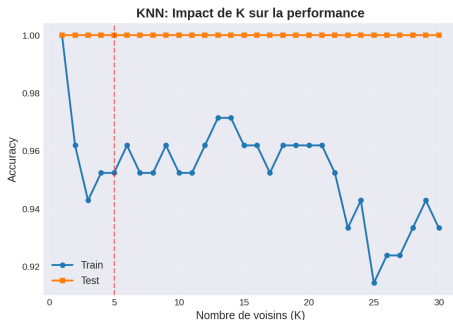
Workflow recommandé

Exploration → Préparation → Plusieurs modèles → Comparaison → Meilleur modèle

Éviter l'Overfitting

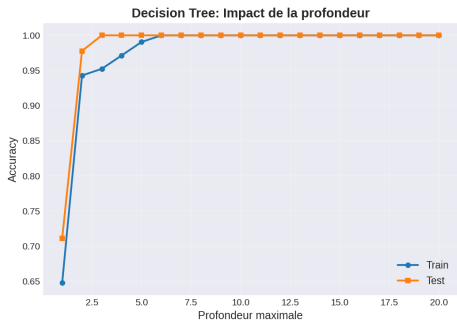
Avec KNN :

- Augmenter K
- Normaliser les features
- Réduire le nombre de features
- Plus de données d'entraînement



Avec Arbres :

- Limiter max_depth
- Augmenter min_samples_split
- Augmenter min_samples_leaf
- Pruning (élagage)



À vous de jouer !

- 1 Chargez le dataset Iris
- 2 Explorez les données (shape, describe, visualisation)
- 3 Divisez en train/test (70/30)
- 4 Entraînez un KNN avec $K=3$
- 5 Entraînez un arbre avec `max_depth=3`
- 6 Comparez les deux modèles (accuracy, confusion matrix)
- 7 Testez différentes valeurs de K et `max_depth`
- 8 Trouvez les meilleurs paramètres
- 9 Visualisez l'arbre final
- 10 Interprétez les résultats

Ce que vous avez appris

- **Classification** : prédire des catégories
- **KNN** : vote des K plus proches voisins
- **Arbres de décision** : série de questions
- **Métriques** : accuracy, précision, rappel, F1
- **Matrice de confusion** : visualiser les erreurs
- **Bonnes pratiques** : normalisation, validation, éviter overfitting